

Input/Output (I/O) and Parsing

*File Handling, Serialization, and Data
Processing*

Learning Objectives

- What We Will Learn
 - Understand the difference between **Byte** and **Character** streams.
 - Read and write text files using java.io and java.nio.
 - Use **Try-With-Resources** for safe stream handling.
 - Understand **Object Serialization** (saving state).
 - Parse structured data (CSV, JSON).
 - Integrate File Choosers into **Swing** applications.
 - Handle IOException properly.

The File Class vs. Path

- **Representing Files**
 - **java.io.File** (Legacy):
 - *Represents a file or directory pathname.*
 - *Limited functionality.*
 - *Used heavily in older Swing components.*
 - **java.nio.file.Path** (Modern - Java 7+):
 - *Represents a path in the file system.*
 - *Richer API (copy, move, delete, attributes).*
 - *Used with the Files utility class.*
 - **Conversion:** `file.toPath()` or `pathToFile()`

```
// Legacy
File oldFile = new File("data.txt");

// Modern
Path newPath = Paths.get("data.txt");
```

Byte vs. Character Streams

- **Stream Hierarchy**
 - **Byte Streams (InputStream / OutputStream):**
 - Handle binary data (images, audio, compiled code).
 - Classes: FileInputStream, FileOutputStream.
 - **Character Streams (Reader / Writer):**
 - Handle text data (handles encoding like UTF-8).
 - Classes: FileReader, FileWriter, BufferedReader.
 - **Rule:** Use **Character Streams** for text files to avoid encoding issues.

Reading Text Files

- Reading Data
 - Modern Approach (java.nio):

```
Path path = Paths.get("config.txt");  
String content = Files.readString(path); // Java 11+  
List<String> lines = Files.readAllLines(path);
```

- Traditional Approach (java.io)

```
try (BufferedReader br = new BufferedReader(new  
FileReader("file.txt"))) {  
    String line = br.readLine();  
}
```

- **Note:** IOException is a **Checked Exception**.

Try-With-Resources

- Managing Resources Safely
- **Problem:** Streams must be closed (close()) to free system resources.
- **Risk:** If an exception occurs, close() might never run.
- **Solution:** Try-With-Resources (Java 7+).
 - Automatically closes resources implementing AutoCloseable.
 - Eliminates need for finally block.

```
// Automatic closing
try (FileWriter writer = new FileWriter("out.txt")) {
    writer.write("Hello");
} catch (IOException e) {
    e.printStackTrace();
}
// writer.close() called automatically here
```

Object Serialization

- **Saving Objects to Disk**
 - **Serialization:** Converting an object into a byte stream to save it.
 - **Requirement:** Class must implement Serializable interface.
 - Marker interface (no methods).
 - **Classes:** ObjectOutputStream, ObjectInputStream.

```
// Saving
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("user.ser"))) {
    oos.writeObject(myUserObject);
}

// Loading
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("user.ser"))) {
    User u = (User) ois.readObject();
}
```

Serialization Best Practices

- **Serialization Risks & Controls**
 - **serialVersionUID:**
 - Unique ID for versioning.
 - If class changes but ID matches, deserialization succeeds.
 - If ID mismatches, `InvalidClassException` occurs.
 - **transient** Keyword:
 - Marks fields that should not be saved (e.g., passwords, temporary caches).
 - Security: Never deserialize untrusted data (security vulnerability).

```
public class User implements Serializable {  
    private static final long serialVersionUID = 1L;  
    private transient String password; // Not saved  
}
```

Introduction to Parsing

- What is Parsing?
 - **Definition:** Analyzing a string of symbols to extract structured data.
 - **Input:** Raw text (CSV, JSON, XML, Log files).
 - **Output:** Java Objects / Collections.
 - **Why?** Files are often stored in standard formats for interoperability.
 - **Common Formats:**
 - **CSV:** Comma Separated Values (Simple).
 - **JSON:** JavaScript Object Notation (Web standard).
 - **XML:** Extensible Markup Language (Legacy/Config).

Parsing CSV

- **Handling CSV Files**
 - **Simple Approach:** `String.split(",")`.
 - Risky if data contains commas.
 - **Better Approach:** Scanner or Libraries (OpenCSV).

```
List<String> users = new ArrayList<>();
try (Scanner sc = new Scanner(new
File("users.csv"))) {
    while (sc.hasNextLine()) {
        String[] data = sc.nextLine().split(",");
        users.add(data[0]); // Get first column
    }
}
```

- **Challenge:** Handling quoted fields (e.g., "Doe, John").

Parsing JSON

- Handling JSON Data
 - **Standard:** Lightweight data-interchange format.
 - **Java Support:** No built-in parser in JDK (until recently).
 - **Libraries:**
 - **Gson** (Google).
 - **Jackson** (Industry Standard).
 - **org.json** (Simple).

```
// Using Gson
Gson gson = new Gson();
User user = gson.fromJson(jsonString, User.class);
String json = gson.toJson(user);
```

I/O in Swing

- File Choosers in GUI
- JFileChooser: Standard dialog for opening/saving files.

```
JFileChooser fc = new JFileChooser();  
int returnVal = fc.showOpenDialog(frame);  
if (returnVal == JFileChooser.APPROVE_OPTION) {  
    File file = fc.getSelectedFile();  
    // Process file  
}
```

- **Filters:** Restrict visible files (e.g., *.txt

```
fc.setFileFilter(new FileNameExtensionFilter("Text  
Files", "txt"));
```

NIO.2 Features

- Modern NIO.2 (java.nio.file)
 - **Files Utility Class:** Static methods for file operations.
 - **Common Operations:**
 - Files.copy(), Files.move(), Files.delete().
 - Files.exists(), Files.isRegularFile().
 - **Walking Trees:** Files.walk(Path) to traverse directories.
 - **Benefits:** Better exception handling, symbolic link support, atomic moves.

```
if (Files.exists(path)) {  
    Files.copy(path, Paths.get("backup.txt"),  
StandardCopyOption.REPLACE_EXISTING);  
}
```

Real-World Context

- Serialization is rarely used for data exchange (too Java-specific), but JSON/CSV is standard for sharing data.
- NIO.2 is the modern standard, but Swing still relies heavily on `java.io.File` objects for `JFileChooser`.

Summary & Lab

- **Summary:**
 - Use Character Streams for text, Byte Streams for binary.
 - Always use Try-With-Resources.
 - Serialization saves Java objects; Parsing reads standard formats.
 - Use JFileChooser for GUI interactions.
- **Lab Exercise:**
 - Add a "Save" and "Load" menu to your Swing App.
 - Save: Serialize the current state (e.g., form data) to a .dat file OR export to .csv.
 - Load: Read the file and populate the form fields.
 - Use JFileChooser to select the file location.